Spiking Neural P Systems with Weights and Delays on Synapses

Yanyan Li, Bosheng Song*, Xiangxiang Zeng

College of Information Science and Engineering, Hunan University, Changsha 410082, Hunan, China

Abstract

Spiking neural P systems with weights (WSN P systems, in short) are neural computing devices with a bio-inspired design, which imitate communications between two neighbors and changes of potentials on cells. In this work, a novel kind of spiking neural-like P systems is investigated, named spiking neural P systems with weights and delays on synapses ($WDSN$ P systems, for short), which offers a simple way to choose the applicable rules of neurons. In $WDSN$ P systems, on each synapse, there exists the weights and the delays, where the weights can be positive and negative. The computation power of $WDSN$ P systems is examined. Results demonstrate that both under the generating and accepting mode, only six neurons are required for $WDSN$ P systems to calculate the set of Turing computable numbers. The *Subset Sum* problem belongs to **NP-Complete** problems can also be solved using $WDSN$ P systems.

Keywords: Spiking neural P systems with weights, Delays on synapses, Membrane computing, Universality, Subset Sum Problems

1. Introduction

Membrane computing is a branch of natural computing that is rapidly growing in popularity. It was first proposed by Academician *Gh. Păun* of the Eu-

*Corresponding author

Email addresses: yanyanli@hnu.edu.cn (Yanyan Li), boshengsong@hnu.edu.cn (Bosheng Song), xzeng@hnu.edu.cn (Xiangxiang Zeng)

ropean Academy of Sciences at the end of 1998 [11]. The goal of membrane computing is to construct models, which imitate the function and structure of living cells. Distributed and parallel devices, sometimes known as P systems, are the models of membrane computing. In P systems, there exists two main types: *cell-like P systems* [22, 24, 26] represented by trees that have hierarchical structures and *tissue-like P systems* [10, 23, 25] or *neural-like P systems* [5, 7, 32] that have graph structures similar to the network. References [12, 19, 21] provide a comprehensive introduction to membrane computing.

A class of neural computing parallel devices, known as neural-like P systems, has recently been rapidly developed (SN P systems, in short) [5]. SN P systems can be represented through the direct graph. Nodes in a direct graph correspond to neurons of SN P systems, and arcs correspond to the synapses between two neighboring neurons, where the movement of impulses (spikes, indicated by the letter a) along the synapse. The structure of SN P systems looks like a network. In one neuron, there exists only one kind of object evolved by the application of spiking rules. There two kinds of spiking rules: spiking rules and forgetting rules. One is the spiking rule $E/a^c \rightarrow a; d$, where E represents the regular expression over a, c ($c \geq 1, c \in \mathbb{N}$) represents the number of consumed spikes, d ($d \geq 0, d \in \mathbb{N}$) represents the delay belongs to each neuron. When k spikes are inserted in a neuron in the following patten: $a^k \in L(E), k \geq c$, c spikes are consumed, a single is created waits for d computational steps. All neurons linked to the neuron receive spikes from the output synapse. The other one is the forgetting rule $a^s \rightarrow \lambda$ ($s \geq 1$) and illustrates that when a neuron has s spikes, s spikes are ignored and nothing is created.

The earliest proposed SN P systems are synchronized because they have a global clock. That is, all neurons belong to the system are active at the same time. In one neuron with applicable rules, all rules can be applied choosing in a non-deterministic manner for each step. Furthermore, an applicable rule must be utilized, with a maximum of one rule being used in each neuron in one step. The computation results of spiking neural P systems have several representations, one general representation is the number of spikes shown in

the output neuron in the halting configuration or the number of spikes sent to the environment by the output neuron during a halting computation, and other representations refer to [5].

There are now many other types of SN P systems, such as anti-spikes exist in spiking neural P systems [8, 16], spiking neural P systems working asynchronously [2], inhibitory synapses exist in spiking neural P systems [27], coupled neural P systems [15, 18], axon P systems [37], neuron division, budding, and separation exist in spiking neural P systems [17], astrocyte-like control exists in spiking neural P systems [1, 9, 13], homogeneous SN P systems [40], asynchronous spiking neural P systems [2, 35], weights exist in spiking neural P systems [31], delay on synapses exists in spiking neural P systems [30], sequential spiking neural P systems [4, 36, 39], exhaustive rule application exists in spiking neural P systems [6, 38], rules on synapse exist in spiking neural P systems [28], colored spikes exist in spiking neural P systems [29], evolution-communication spiking neural P systems [33], numerical spiking neural P systems [34], nonlinear spiking neural P systems [14] and so on.

In [31], Wang et al. proposed spiking neural P systems with weights (WSN P systems, for brevity). In WSN P systems, each neuron spikes when its potential is equal to the given firing threshold potential, several potentials (spikes) are consumed and one potential is produced at that time. Then this unit potential multiplied by the weights of synapses will be transmitted to all neurons connected with the spiking neuron. In [30], Song et al. introduced spiking neural P systems with delay on synapses (DSN P systems, for brevity). Since axons possess various lengths, spikes leave a presynaptic neuron to reach the postsynaptic neurons at different computational steps. Inspired by this biological phenomenon, the DSN P system was proposed. DSN P systems have a unique feature that the separation between the spiking rules in neurons and the delays on synapses, so the postsynaptic neurons can receive the same number of spikes at different computational steps.

In this article, combining features of WSN P systems and DSN P systems, a novel type of SN P system is proposed, which is called spiking neural P

Table 1: The comparison between WSN P systems and $WDSN$ P systems, where $\mathbb{Z}$ represents the set of integers, $*$ represents unbounded on parameters, and 2, acc represent systems working under the generating mode and the accepting mode, respectively.

Turing Universality	WSN P systems	$WDSN$ P systems
In the generating mode	$N_2W_{\mathbb{Z}}SNP_* = NRE$ [31]	$N_2W_{\mathbb{Z}}DSNP_6 = NRE$
In the accepting mode	$N_{acc}W_{\mathbb{Z}}SNP_* = NRE$ [31]	$N_{acc}W_{\mathbb{Z}}DSNP_6 = NRE$

systems with weights and delays on synapses ($WDSN$ P systems, for brevity). $WDSN$ P systems not only satisfy the advantages of WSN P systems and DSN P systems but also provide a simple way to choose the applicable spiking rules. We consider each neuron in $WDSN$ P systems to have one potential, unlike SN P systems. Spiking rules of neurons are $E/a^c \rightarrow a$, where each synapse has weights and delays. The spiking rules of each neuron are applied with a non-deterministic manner. When a spiking rule of the neuron is applied, one potential of the cell (neuron) multiplies with the weight on the synapse and then it is transmitted to all cells (neurons) are connected through the cell associated with the spiking rule. The weight on the synapse belongs to the set of real numbers, so it includes the positive weight and the negative weight. If the delay t ($t > 0$) exists on synapses, the emitted potential will be transmitted to neighboring cells (neurons) after t computational steps. The computational result of $WDSN$ P systems, where the first and second spikes are broadcast to the environment, is defined as the number of steps between the first and second consecutive spikes. We also take into account $WDSN$ P systems that operate in both the generating and accepting modes. Compared with [31], the number of cells used in $WDSN$ P systems is less than the used in WSN P systems. Table 1 shows the comparison.

The following is the structure of the rest of this article. Section 2 presents some preliminaries. Section 3 shows a description of the model under investigation in this study. Section 4 studies the computation power of $WDSN$ P systems. Section 5 investigates how $WDSN$ P systems solve the *Subset Sum* problem. Section 6 makes a summary and presents some open problems.

2. Preliminaries

It is useful for readers to know the basic knowledge about membrane computing [20] and formal language and automata theory [3]. Here, we will introduce some of the necessary prerequisites and the definitions that need to be used in subsequent proofs.

In mathematics, the set of natural numbers is represented by $\mathbb{N}$, the set of integer numbers is represented by $\mathbb{Z}$, the set of national numbers is represented by $\mathbb{Q}$ and the set of computable real numbers is represented by $\mathbb{R}_c$. The finite set of all strings of symbols is denoted as an alphabet V. $V - \{\lambda\}$ represents the finite set of all non-empty strings of symbols, where the empty string is λ. We write a^* or a^+ instead of $\{a^*\}$ or $\{a^+\}$ under $V = \{a\}$ is a singleton.

For the universality proofs, we verify whether the proposed system generates NRE (the family of sets of Turing computable numbers) or $SLIN$ (the family of sets of semilinear computable numbers). To complete the proofs of universality, we define a register machine $M = (m, H, l_0, l_h, R)$, where m denotes the number of registers, H represents the sets of labels of instructions, l_0 is the start instruction, l_h is the halt instruction and R represents the sets of instructions. Each instruction of M has a unique label. M includes instructions with three following forms:

- $l_i : (ADD(r), l_j, l_k)$ indicates an ADD instruction that increases the number 1 to the specified register r and go to the instruction l_j or l_k;
- $l_i : (SUB(r), l_j, l_k)$ indicates a SUB instruction. When the content of the specified register r is non-empty, then decreases the number 1 from r and go to the instruction l_j; otherwise, go to the instruction l_k;
- $l_i : HALT$, a $HALT$ instruction, the halt instruction can end the computation of the system.

M (the register machine) starts from the start (initial) instruction. Initially, all registers are empty except that the specified is non-empty. Then, M will continue to use instructions illustrated by labels. The computation will be

terminated if M reaches the halt instruction. Meanwhile, the number n placed in the first register represents the result of the computation. $N(M)$ denotes the set of all numbers computed by M. Since M possesses Turing universality, they can compute all sets of Turing computable numbers. In other words, they can characterize NRE.

In general, we assume that l_0 labels the initial instruction. All registers are empty besides the first one under the halting configuration, and the output register's contents are never decremented during calculation.

In addition to the generating mode, M also can be used under the accepting mode. Initially, the first register stores a number and all others are empty under the accepting mode. If one computation begins with the initial configuration and finishes with the halting, the system will accept a number and store it in the first register. Of course, if we use ADD instructions $l_i : (ADD(r), l_j)$ with $l_j = l_k$, then M can obtain all sets of computable numbers in NRE.

For convenience, we give a convention as follows. Suppose there exists one generating device H_1 and one accepting device H_2. We disregard the number 0 when comparing the power of two devices. In other words, if $N(H_1) - \{0\} = N(H_2) - \{0\}$, then we write $N(H_1) = N(H_2)$. This phenomenon relates to the case where empty string are ignored in the ordinary practice in the formal languages and automata theory.

3. Spiking neural P systems with weights and delays on synapses

In this section, we will introduce the definition of spiking neural P systems with weights and delays on synapses ($WDSN$ P systems, for short).

Definition 1. *An SN P system with weights and delays on synapses (a $WDSN$ P system, for brevity), of degree $m \geq 1$, is a build of the form*

$$\Pi = (\sigma_1, \ldots, \sigma_m, syn, in, out),$$

where

- $\sigma_1, \ldots, \sigma_m$ *are* m *neurons, where* $\sigma_i = (p_i, R_i), 0 < i \leq m$. $p_i \in \mathbb{R}_c$ *is the initial potential of* σ_i. R_i *represents the finite set of spiking rules* $T_i/d_s \to 1, s = 1, 2, \ldots, n_i$, $n_i \geq 1$. *For the spiking rules, the firing threshold potential of* σ_i *is denoted as* T_i *(*$T_i \in \mathbb{R}_c, T_i \geq 1$*), the number of consumed spikes is denoted as* $d_s \in \mathbb{R}_c$ *(*$0 < d_s \leq T_i$*);*

- $syn \subseteq \{1, 2, \ldots, m\}^2 \times \mathbb{R}_c \times \mathbb{Z}$ *represents the set of synapses between two neighbor neurons. For each synapse* $(i, j) \in \{1, 2, \ldots, m\}^2$*, at most one synapse* (i, j, r, t) $(i \neq j, r \neq 0, t \neq 0)$ *belongs to the set* syn*, where* r *shows the weight on the synapse* (i, j) *and* t *shows the delay on the synapse* (i, j)*;*

- *in is the input neuron of the system;*

- *out is the output neuron of the system.*

The following is a strategy of how to apply spiking rules. Suppose that σ_i includes potential p at a given time. When $p = T_i$, the spiking rule $T_i/d_s \to 1$ is applied, and d_s potentials are consumed, then neuron σ_i will emit one potential to all neighbors σ_j which satisfies $(i, j, r, t) \in syn$. At this time, there exists $T_i - d_s$ potentials in neuron σ_i. On each synapse $(i, j, r, t) \in syn$, if $r \neq 0$, then σ_i will emit r potentials to all neighbor neurons σ_j. The potential of σ_j will become the existing potential adds the receiving potential. Know that $r \in \mathbb{R}_c$, the potential of neuron σ_j can be increased or decreased depending on wether r is positive or negative. Meanwhile, if $t \neq 0$, the potential will be emitted to σ_j after t delays. Of course, suppose neuron σ_i spikes and the system without the output neuron, the system will lose the potential spiked by neuron σ_i.

Considering the threshold firing mechanism of spiking rules, we specify several conditions as follows.

- for every neuron σ_i, there exists a fixed firing threshold potential T_i;

- when σ_i includes the potential equals to the firing threshold potential, all spiking rules of σ_i can be used, and at most one spiking rule is chosen non-deterministically;

- when σ_i spikes, there is only one potential to be emitted.

Suppose σ_i has potential p at a given time, and the firing threshold potential is T_i. When $p = T_i$, all spiking rules associated with σ_i can be used. When $p < T_i$, the potential of σ_i will be reset to 0. When $p > T_i$, the potential of σ_i will remain unchanged. For clarity, we express the spiking threshold firing mechanism with the formulation as follows.

For the neuron σ_i, the initial potential is p. At step t, if it receives k potentials from other neurons, then the potential of σ_i is p' at step $t+1$, where

$$p' = \begin{cases} k & when\ p < T_i; \\ p - d_s + k & when\ p = T_i\ ,\ rule\ T_i/d_s \to 1\ can\ be\ used; \\ p + k & when\ p > T_i. \end{cases}$$

Generally speaking, there exists a global clock in $WDSN$ P systems, so activations of all neurons are synchronized. In other words, in the view of the system, all neurons work parallelly. At one step, each neuron at most applies one spiking rule, and it is chosen from all applicable spiking rules nondeterministically.

Assume that in a $WDSN$ P system Π, the distribution of neurons' potentials composes the configuration of Π. The initial configuration is described by the tuple $< p_1, p_2, \ldots, p_m >$, where m shows the number of cells. In this way, we define the transition between two consecutive configurations. The system's computation reveals that any sequence of transitions begins with the initial configuration and ends with halting. When the system reaches a halting configuration and no spiking rules are available, the computation comes to a halt. For any computation, the system will generate or accept a spiking train, which is a binary sequence. In this spiking train, 1 represents the output neuron emitting one potential (one spike), and 0 represents the output neuron that does not spike.

For the $WDSN$ P system, the interval between two moments t_1 and t_2 represents the result of a computation. At t_1, the system emits the first spike.

At t_2, the system emits the second spike. We write $t_2 - t_1$ to represent the result of a computation, and we say Π computes this number. In this way, we use $N_2(\Pi)$ to represents the set of all numbers computed by Π, where the distance between the first and second consecutive spikes emitted by the output cell is represented by the subscript 2. Note that 0 can not be computed, which is why we ignore 0 when we prove the computation power of the system.

Spiking neural P systems with weights and delays on the synapse also can be worked under the accepting mode. Under the accepting mode, the system begins with the initial configuration. Then at steps t_1, t_2, one spike is introduced to the input cell, respectively. When the computation halts, the number $t_2 - t_1$ is accepted by the system. $N_{acc}(\Pi)$ denotes the set of numbers accepted by the system Π.

The input cell (neuron) σ_{in} is ignored in the generating mode, the output cell (neuron) σ_{out} is ignored in the accepting mode. In the following proofs, we use σ_i instead of "neuron/cell i" .

We use $N_\alpha W_X D_t SNP_m$ denotes the family of all sets $N_\alpha(\Pi)$ ($\alpha \in \{2, acc\}$), which is calculated by $WDSN$ P systems with at most $m \geq 1$ neurons, $X \in \{\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}_c\}$ includes using weights, the firing threshold potentials and the number of consumed potentials in spiking rules, and $t \in \{\mathbb{Z}\}$ represents the delay on synapses between two neighbor neurons. When the number of neurons has no bound, $*$ replaces the subscript m.

4. Turing Universality of WDSN P systems with Integers

In this section, we will investigate the Turing universality of $WDSN$ P systems. Before we give the proof of Turing universality about $WDSN$ P systems, we will give one lemma likes [31].

Lemma 4.1. *In number theory,* $\mathbb{N} \subseteq \mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R}_c$, *so it can obtain that* $N_\alpha W_{\mathbb{N}} DSNP_m \subset N_\alpha W_{\mathbb{Z}} DSNP_m \subset N_\alpha W_{\mathbb{Q}} DSNP_m \subset N_\alpha W_{\mathbb{R}_c} SNP_m = NRE$ *with all* $\alpha \in \{2, acc\}$, $m \leq 1$ *or* $m = *$.

In what follows, $WDSN$ P systems are illustrated to be Turing universality both under the generating and accepting configuration, where parameters "weight" and "delay" are integers.

Theorem 4.1. $N_2W_{\mathbb{Z}}DSNP_6 = NRE$.

Proof. In the generating mode, $WDSN$ P systems have the ADD, SUB and $OUTPUT$ module. We only need to prove the inclusion $NRE \subseteq N_2W_{\mathbb{Z}}DSNP_6$.

Considering M (a register machine) likes section 2. All registers belong to M are empty besides the first one in the halting configuration, and the content of the output register is never decremented during calculating. We build a $WDSN$ P system Π, which has ADD, SUB and $OUTPUT$ modules that simulate the instructions belonging to M. In Π, each register r relates to a neuron σ_r. The register r has the number r represents that σ_r possesses $2n+2$ potentials. Each label $l_i \in H$ associates with a neuron σ_{l_i}, also we consider some auxiliary neurons $\sigma_{l_i^{(j)}}$ with $j = 1, 2, \ldots$.

The modules will be presented in the form of a graph. Under the initial configuration, contents of all neurons are empty, but the initial neuron σ_{l_0} includes potential 1, and neurons relate to registers include potential 2. For spiking rules, when σ_{l_i} ($l_i \in H$) includes potential 1, σ_{l_i} will be activated and spiking rules associated with σ_i will be applied. So the module begins to work, simulating the relevant instructions belong to M.

- *Module ADD*: to simulate an ADD instruction $l_i : (ADD(r), l_j, l_k)$.

Module ADD shows in Fig.1 with six neurons: neuron σ_r associates with the register r, neurons $\sigma_{l_i}, \sigma_{l_j}, \sigma_{l_k}$ relate to instructions l_i, l_j, l_k and two auxiliary neurons $\sigma_{l_i}^{(1)}$, $\sigma_{l_i}^{(2)}$.

The ADD module starts with the starting instruction l_0. Assume that at step t, neuron σ_{l_i} includes one potential, but others are empty except those neurons related to registers. With the potential 1, neuron σ_{l_i} becomes active, where the spiking rule $1/1 \to 1$ is applied and produces one potential. Meanwhile, σ_r

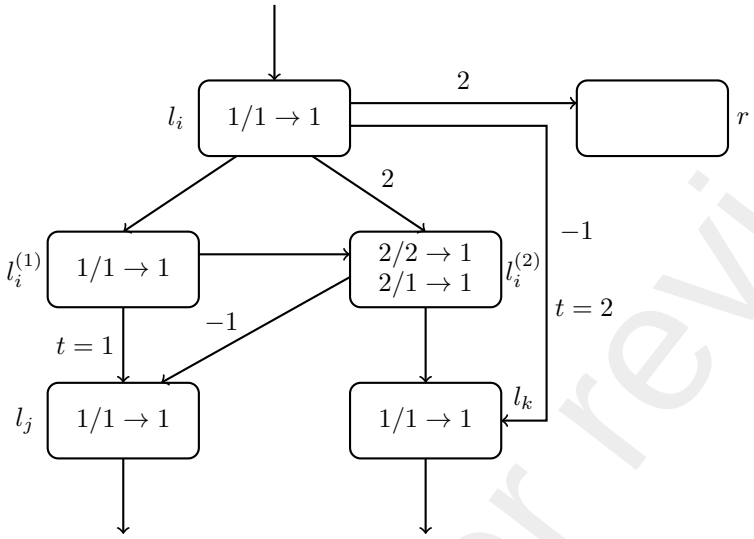

Figure 1: The ADD module of $WDSN$ P systems for performing the ADD instruction under the generating case.

accepts potential 2 (the content of register r is increased 1), $\sigma_{l_i}^{(2)}$ accepts potential 2, $\sigma_{l_i}^{(1)}$ accepts potential 1. Since the synapse $(\sigma_{l_i}, \sigma_{l_k})$ has a delay of 2 , neuron σ_{l_k} will receives potential -1 after two steps. At step $t+1$, $\sigma_{l_i}^{(1)}$ includes one potential, $\sigma_{l_i}^{(2)}$ has 2 potentials. So in neuron $\sigma_{l_i}^{(1)}$, spiking rule $1/1 \to 1$ can be used, then it produces one potential and this potential will be transmitted to neuron $\sigma_{l_i}^{(2)}$. Since the synapse $(\sigma_{l_i}^{(1)}, \sigma_{l_j})$ has a delay of 1, neuron σ_{l_j} will receive a potential after one step. In neuron $\sigma_{l_i}^{(2)}$, it has potential 2 and two spiking rules associated with neuron $\sigma_{l_i}^{(2)}$ can be applied non-deterministically, so it has two cases as follows.

- The spiking rule $2/2 \to 1$ can be used. In neuron $\sigma_{l_i}^{(2)}$, two potentials are consumed and one potential is produced. At step $t+2$, $\sigma_{l_i}^{(2)}$ accepts potential 1 from $\sigma_{l_i}^{(1)}$, since potential 1 is less than the firing threshold potential 2 of $\sigma_{l_i}^{(2)}$, the potential of $\sigma_{l_i}^{(2)}$ is reset to 0. Also, σ_{l_k} accepts potential 1 from $\sigma_{l_i}^{(2)}$ and potential -1 from σ_{l_i}, where -1 equals potential 1 of σ_{l_i} multiplies the weight -1 on the synapse $(\sigma_{l_i}, \sigma_{l_k})$. So the potential in σ_{l_k} is 0. Meanwhile, at the same time, σ_{l_j} accepts potential -1 from

$\sigma_{l_i}^{(2)}$. Since potential -1 is less than the firing threshold potential -1, the potential of σ_{l_j} is reset to 0. At step t+3, since the potential 1 is emitted from $\sigma_{l_i}^{(1)}$ to σ_{l_j}, the spiking rule $1/1 \to 1$ can be used and it starts to simulate the instruction l_j.

- The spiking rule $2/1 \to 1$ can be used. $\sigma_{l_i}^{(2)}$ consumes one potential and generates one potential. At step t+2, $\sigma_{l_i}^{(2)}$ accepts potential 1 from $\sigma_{l_i}^{(1)}$ and the potential of $\sigma_{l_i}^{(2)}$ becomes 2. Meanwhile, potential -1 is transmitted from $\sigma_{l_i}^{(2)}$ to σ_{l_j}. Since potential -1 is less than the firing threshold potential 1, the potential of σ_{l_j} resets to 0. Also, σ_{l_k} accepts potential 1 from neuron $\sigma_{l_i}^{(2)}$ and receives potential -1 from σ_{l_i} with the weight -1 on the synapse $(\sigma_{l_i}, \sigma_{l_k})$, so the potential in σ_{l_k} is 0. At step $t+3$, $\sigma_{l_i}^{(2)}$ has potential 2, so the spiking rule $2/1 \to 1$ can be used again and potential -1 is transmitted to neuron σ_{l_j}. Meanwhile, σ_{l_j} accepts potential 1 from $\sigma_{l_i}^{(1)}$, so the potential of σ_{l_j} is 0. Also, σ_{l_k} accepts potential 1 from $\sigma_{l_i}^{(2)}$ again. So the spiking rule $1/1 \to 1$ can be used and it simulates the instruction l_k.

.

From the above description, we can see that the $WDSN$ P systems we constructed can perform the ADD instruction of M correctly.

- *Module SUB*: to simulate a SUB instruction $l_i : (SUB(r), l_j, l_k)$.

The SUB module is shown in Fig.2 and is composed of six neurons: neuron σ_r associates with the register r, neurons $\sigma_{l_i}, \sigma_{l_j}, \sigma_{l_k}$ associate with instructions l_i, l_j, l_k, and two auxiliary neurons $\sigma_{l_i}^{(1)}$, $\sigma_{l_i}^{(2)}$.

The SUB module starts with the starting instruction l_0. Initially, all registers are empty except register r. Suppose that at step t, σ_{l_i} includes one potential, so the spiking rule $1/1 \to 1$ can be applied. Since the weight on the synapse (σ_{l_i}, σ_r) is -1, potential -1 is transmitted to register r. Meanwhile, $\sigma_{l_i}^{(1)}$ receives one potential from σ_{l_i}. For σ_r, it includes two cases as follows.

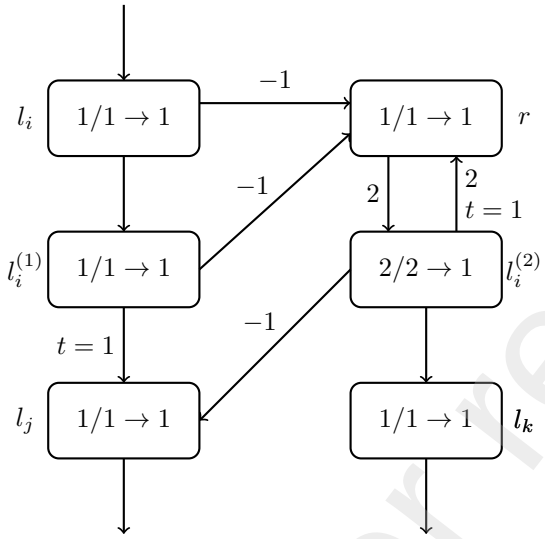

Figure 2: The SUB module of $WDSN$ P systems for performing the SUB instruction under the generating case.

- Neuron σ_r has potential $2n + 2(n \geq 1)$ and it corresponds to a case that the number stored in σ_r is greater than 0. At step t, the potential of σ_r is $2n + 1$. Since $2n + 1$ is greater than the firing threshold potential 1, the potential of σ_r remains unchanged. At step $t + 1$, $\sigma_{l_i}^{(1)}$ includes the potential 1, so the spiking rule $1/1 \to 1$ can be used. Since the weight on the synapse $(\sigma_{l_i}^{(1)}, \sigma_r)$ is -1, σ_r accepts potential -1 and the potential of σ_r is $2n$. This indicates that the number contained in r has decreased by one. At step $t + 2$, cell σ_{l_j} accepts potential 1 from $\sigma_{l_i}^{(1)}$ and it simulates the instruction l_j.

- Neuron σ_r has potential 2 and it relates to a situation in which the number placed in σ_r is 0. At step t, σ_r accepts potential -1 from σ_{l_i} and the potential of σ_r is 1, so the spiking rule $1/1 \to 1$ can be applied. At step $t + 1$, since the weight on the synapse $(r, \sigma_{l_i}^{(2)})$ is 2, $\sigma_{l_i}^{(2)}$ receives potential 2. Meanwhile, σ_r accepts potential -1 from $\sigma_{l_i}^{(1)}$, so the potential of σ_r is 0. At step $t+2$, $\sigma_{l_i}^{(2)}$ includes potential 2, so the spiking rule $2/2 \to 1$ can be applied and σ_{l_j} receives potential -1 from $\sigma_{l_j}^{(2)}$. At the same time, since

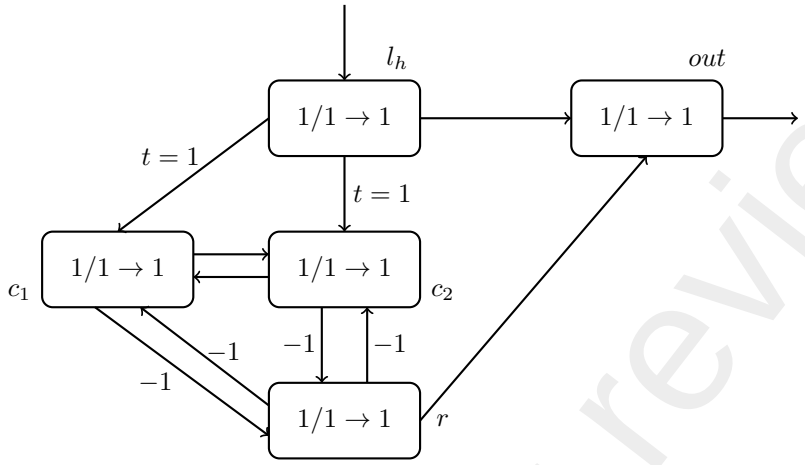

Figure 3: The module $OUTPUT$ for outputting the results of computation.

the delay on the synapse $(\sigma_{l_i}^{(1)}, \sigma_{l_j})$ is 1, σ_{l_j} receives potential 1 from $\sigma_{l_i}^{(1)}$, so the potential of σ_{l_j} is 0. Since the delay on the synapse $(\sigma_{l_i}^{(2)}, \sigma_r)$ is 2, σ_r accepts potential 2 from $\sigma_{l_i}^{(2)}$, it demonstrates that r still has a value of 0. Also, σ_{l_k} accepts potential 1 from $\sigma_{l_i}^{(2)}$ and the spiking rule $1/1 \to 1$ can be applied, so it simulates the instruction l_k.

From the above description, we can find that Π can simulate the SUB instruction correctly.

- *Module* $OUTPUT$: output the results of computation.

The $OUTPUT$ component shows in Fig.3. It is composed of five neurons: neuron σ_{l_h} associates with the halting register, neuron σ_{out} associates with the output register, σ_r associates with the register r and two auxiliary neurons σ_{c_1} and σ_{c_2}. Initially, all neurons are empty except σ_r and σ_r has potential $2n+2$. When M halts the computation, the halting instruction is reached. And assume that at step t, σ_{l_h} includes potential 1, so the spiking rule $1/1 \to 1$ can be used and σ_{out} receives potential 1 from σ_{l_h}. The delay on the synapse $(\sigma_{l_h}, \sigma_{c_1})$ and $(\sigma_{l_h}, \sigma_{c_2})$ is 1, respectively. Then at step $t+1$, σ_{c_1} accepts potential 1 from σ_{l_h} and σ_{c_2} accepts potential 1 from σ_{l_h}. At the same time, σ_{out} emits one

potential. It is the first time to emit. At step $t+2$, in σ_{c_1}, it has one potential, so the spiking rule $1/1 \to 1$ can be applied. Then σ_{c_2} accepts potential 1 from σ_{c_1}. Since the weight on the synapse (σ_{c_1}, σ_r) is -1, neuron σ_r accepts potential -1 from neuron σ_{c_1}. In neuron σ_{c_2}, it has one potential and the spiking rule $1/1 \to 1$ can be used, so σ_{c_1} receives potential 1 from σ_{c_2}. Since the weight on the synapse (σ_{c_2}, σ_r) is -1, σ_r accepts potential -1 from σ_{c_2}. Then, the potential of σ_r becomes $2n$. At step $t+3$, σ_{c_1} includes potential 1 and σ_{c_2} has potential 1, so it repeats the process of step $t+2$ until the potential of register r decreases to 1. At step $t+n$, σ_r includes one potential, so the spiking rule $1/1 \to 1$ can be used. In this time, the weight on the synapse (σ_r, σ_{c_1}) and (σ_r, σ_{c_2}) is -1, respectively. Then, neuron σ_{c_1} accepts potential -1 from σ_r and the potential of σ_{c_1} becomes 0. Meanwhile, neuron σ_{c_2} accepts potential -1 from σ_r and the potential of σ_{c_2} becomes 0. Meanwhile, σ_{out} accepts potential 1 from σ_r. At step $t+n+1$, neuron σ_{out} has one potential, so the spiking rule $1/1 \to 1$ can be used and it emits one potential. It is the second time to emit. So the computation result is denoted by the interval between the first and second consecutive spikes are emitted by the output neuron,, that is, $t+n+1-(t+1) = n$.

From above description, we can see that Π outputs the computational results n correctly.

To sum up, M (the register machine) simulates Π correctly. $N_2(M) = N_2(\Pi)$ and the proof complete. □

Theorem 4.2. $N_{acc}W_{\mathbb{Z}}DSNP_6 = NRE$.

Proof. We build a $WDSN$ P system Π and it also can perform under the accepting mode. Similar to *Theorem* 4.1, We build a register machine M, which is performed by the system Π. The system Π includes three components: ADD, SUB and $INPUT$ modules, which simulates the ADD, SUB and $INPUT$ instructions, respectively. Since the ADD instructions of M can be assumed to be deterministic, the ADD modules simulate the ADD instructions with the form of $l_i : (ADD(r), l_j)$. We have the following description of how these three modules perform.

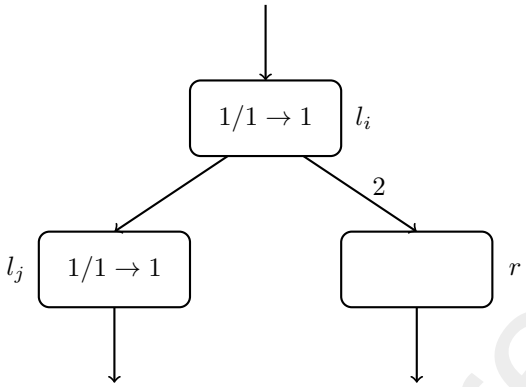

Figure 4: The ADD module of $WDSN$ P systems for performing the ADD instruction under the accepting case.

- *Module ADD*: to simulate an ADD instruction $l_i : (ADD(r), l_j)$.

The ADD component shows in Fig.4. Assume that in step t, neuron σ_{l_i} includes one potential, so the spiking rule $1/1 \to 1$ can be used. Since the weight on the synapse (σ_{l_i}, σ_r) is 2, neuron σ_r accepts potential 2 from σ_{l_i} and it illustrates that the number stored in register r adds 1. σ_{l_j} receives potential 1 from σ_{l_i}. At step $t+1$, σ_{l_j} includes one potential and the spiking rule $1/1 \to 1$ can be used, so it performs the instruction l_j.

- *Module SUB*: to simulate a SUB instruction $l_i : (SUB(r), l_j, l_k)$.

For the SUB instruction $l_i : (SUB(r), l_j, l_k)$, we use the construction likes Fig.2 and we omit the description here.

- *Module INPUT*: input a spiking train into the $WDSN$ P system.

The $INPUT$ component shows in Fig.5, and at step t, the first one potential entering σ_{i_0} and the spiking rule $1/1 \to 1$ can be used, so neuron σ_{a_1} accepts potential 1 from σ_{i_0}, σ_{a_2} accepts potential 1 from neuron σ_{i_0}. At step $t+1$, neuron σ_{a_1} includes one potential and σ_{a_2} has one potential, so the spiking rules $1/1 \to 1$ associate with neurons σ_{a_1} and σ_{a_2} can be used. Neuron σ_{a_1} and neuron σ_{a_2} emit potential with each other. At step $t+2$, σ_{a_1} includes potential

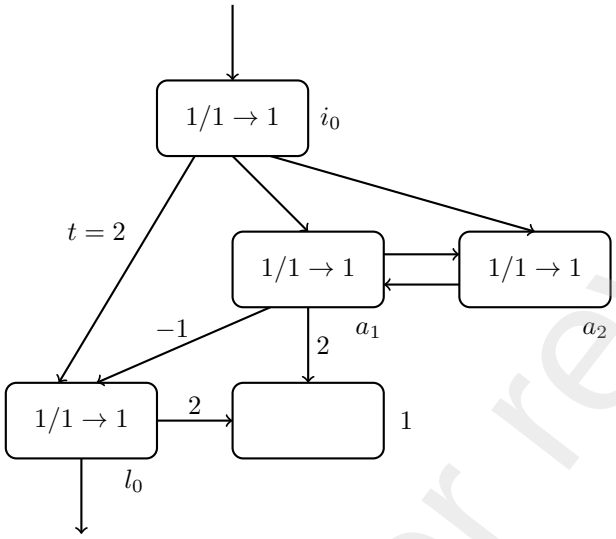

Figure 5: The module $INPUT$ for inputting a spiking train into the $WDSN$ P system.

1, so the spiking rule $1/1 \to 1$ can be applied and σ_{l_0} accepts potential -1 from σ_{a_1}, σ_1 accepts potential 2 from σ_{a_1} and σ_{a_2} receives potential 1 from σ_{a_1}. At the same time, since the delay on the synapse $(\sigma_{i_0}, \sigma_{l_0})$ is 2, σ_{l_0} accepts potential 1 from σ_{i_0}. Then the potential of σ_{l_0} becomes 0. At step $t + 2n + 1$, when the second potential entering the neuron σ_{i_0}, neurons σ_{a_1} and σ_{a_2} receive potential 1 from σ_{i_0} at step $t+2n+2$, respectively. So at step $t+2n+2$, neurons σ_{a_1} and σ_{a_2} have 2 potentials, respectively. Since potential 2 is greater than the firing threshold potential 1, the potential of neurons σ_{a_1} and σ_{a_2} remain unchanged and the spiking rules $1/1 \to 1$ can not be used. At step $t + 2n + 3$, σ_{l_0} accepts potential 1 from σ_{i_0}. At step $t + 2n + 4$, in σ_{l_0}, the spiking rule $1/1 \to 1$ can be used and neuron σ_1 accepts potential 2 from σ_{l_0}. The snap between the first second potentials denotes the number accepted by the $INPUT$ module, which is $t + 2n + 4 - (t + 2) = 2n + 2$.

From above discussion, we achieve that the $WDSN$ P system Π can perform M correctly and the proof completes. □

5. Semiuniform Solution to Subset Sum

The effectiveness of $WDSN$ P systems will be investigated in this section. It shows that $WDSN$ P systems give the semi-uniform solution of the Subset-sum. First, let us review the **NP**-Complete problem: the Subset Sum problem.

Problem:

- Name: Subset Sum Problem
- Instance: A multiset $V = \{v_1, v_2, \ldots, v_n\}$, each element of V is a positive number, and given a positive integer denoted by number S.
- Question: Whether there exists a multiset subset $B \subseteq V$ such that

$$\sum\nolimits_{b \in B} b = S \ ?$$

The construction of $WDSN$ P systems for solving the Subset Sum problem shows in Fig.6. Initially, the potential of neuron σ_{a_i} is 1, the potential of neuron σ_{b_i} is 1, the potential of σ_{c_i} is 2, the potential of neuron σ_i is $v_i + 1$. At step 1, the potential in neuron σ_{a_i} is 1, the spiking rule $1/1 \to 1$ can be used and one potential can be transmitted to σ_{d_i} after one step. The potential in neuron σ_{b_i} is 1, the spiking rule $1/1 \to 1$ can be used and one potential can be transmitted to σ_{c_i}. The potential in neuron σ_{c_i} is 2, the spiking rules $2/2 \to 1$ and $2/1 \to 1$ can be used and we choose one rule to apply nondeterministically. So, it has two cases as follows:

- $2/2 \to 1$ can be used. At step 1, in neuron σ_{c_i}, two potential are consumed and neuron σ_{d_i} receives one potential from σ_{c_i}. Since the received potential is less than the firing threshold potential 2, the spiking rule $2/2 \to 1$ can not be used and the potential in neuron σ_{d_i} is reset to 0. σ_{c_i} accepts one potential from σ_{b_i}. Since the received potential is less than the firing threshold potential 2, the spiking rules $2/2 \to 1$ and $2/1 \to 1$ can not be used and the potential in neuron σ_{c_i} is reset to 0. At step 2, neuron σ_{d_i} receives a potential from σ_{a_i}. Since the received potential is less than

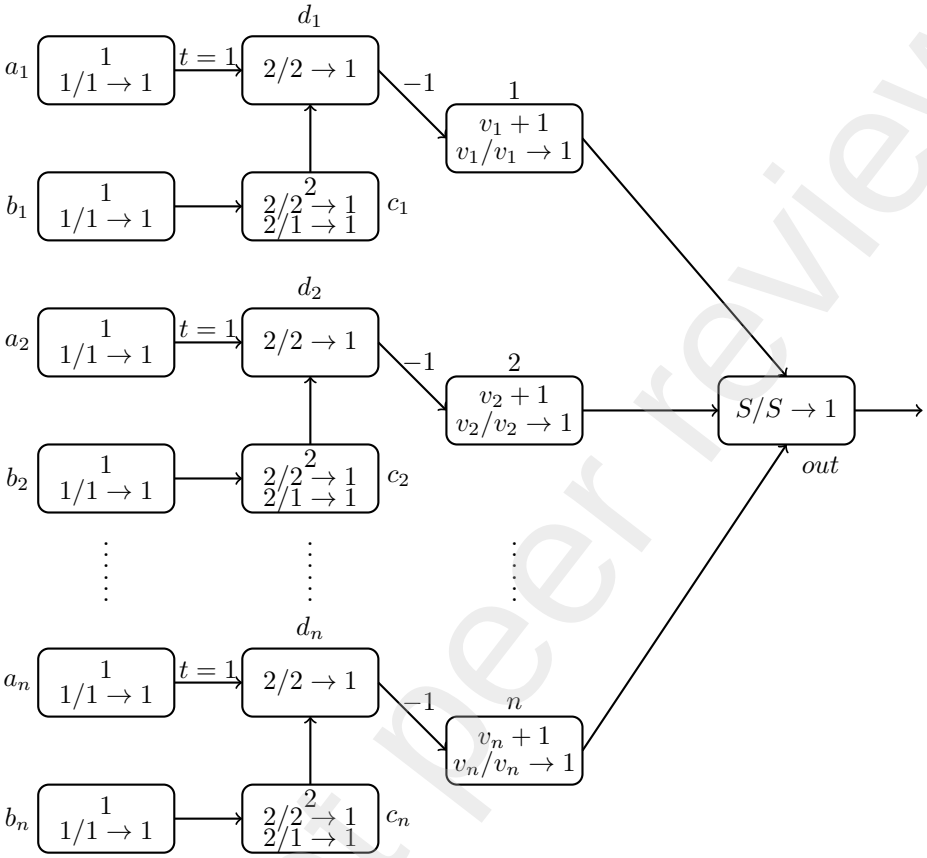

Figure 6: The construction of $WDSN$ P systems for solving the Subset Sum problem.

the threshold potential 2, the spiking rule $2/2 \to 1$ can not be used and the potential in neuron σ_{d_i} is reset to 0. This phenomenon relates to the choosing number v_i not appearing in the subset B.

- $2/1 \to 1$ can be used. At step 1, in neuron σ_{c_i}, one potential is consumed and the remained potential is 1, σ_{d_i} receives one potential from σ_{c_i}. Since the received potential 1 is less than the firing threshold potential 2, the spiking rule $2/2 \to 1$ can not be used and the potential in neuron σ_{d_i} is reset to 0. σ_{c_i} accepts one potential from σ_{b_i}, so the potential in σ_{c_i} is 2. At step 2, the potential in σ_{c_i} is 2, the spiking rule $2/1 \to 1$ can be used, σ_{d_i} receives one potential from neuron σ_{c_i}, and σ_{d_i} accepts potential 1 from

σ_{a_i}, so the potential in σ_{d_i} is 2. At step 3, the potential of neuron σ_{d_i} is 2, the spiking rule $2/2 \to 1$ can be used. With the weight on the synapse (σ_{d_i}, σ_i) is -1, neuron σ_i accepts potential -1 from σ_{d_i}, so the potential in σ_i is v_i. At step 4, in σ_i, the spiking rule $v_i/v_i \to 1$ can be used, σ_{out} accepts potential 1 from σ_i. This case corresponds to the choosing number v_i that appears in the subset B. If the sum of potentials in n channels equals the positive number S, then the spiking rule $S/S \to 1$ can be used and neuron σ_{out} emits one potential. Consequently, one multi-subset B satisfies that the sum of every element of B equals S.

From the above description, the choosing numbers $v_i \in V$ $(i = 1, 2, \ldots, n)$ and S are positive integers. And in general, the systems not only work for positive integers, but also work for real numbers. So how to build a new $WDSN$ P system that solve the Subset Sum problem remains open.

6. Conclusions and Future Works

We presented a new form of spiking neural P systems called $WDSN$ P systems. In $WDSN$ P systems, neurons contain potentials and spiking rules. Each synapse between two neighboring neurons has weights and delays, where the weight can be positive or negative and the delay belongs to the set of positive integers. Besides, in one neuron, the spiking rules have the firing threshold potential, and when the number of potentials contained in the neuron equals the threshold, the spiking rules can be used. In this way, the computation result of $WDSN$ P systems is defined by the steps elapsed between the first and second consecutive spikes which leave the output neuron. The computation power of $WDSN$ P systems was investigated and it can be proved that $WDSN$ P systems with only six neurons are enough to compute the Turing computable numbers. $WDSN$ P systems also can solve the **NP**-Complete problem that the *Subset Sum* problem.

In the future, we attempt to move the delay into the neurons, so the form of spiking rules becomes $E/a^c \to 1; d, c \geq 1, d \geq 1$. And we can introduce some

other strategies into $WDSN$ P systems. For example, $WDSN$ P systems with communications by request. It could be of interest to explore the possibilities computational completeness and efficiency as types of $WDSN$ P systems.

Acknowledgements

The work was supported by the National Natural Science Foundation of China (61972138, 62122025, 62102140, 61872309); Hunan Provincial Natural Science Foundation of China (2020JJ4215, 2021JJ10020); the Key Research and Development Program of Changsha (kq2004016); Open Research Projects of Zhejiang Lab (2021RD0AB02).

References

[1] A. Binder, R. Freund, M. Oswald and L. Vock. Extended spiking neural P systems with excitatory and inhibitory astrocytes. Proceedings of the Fifth Brainstorming Week on Membrane Computing, Sevilla, 2007: 63–72.

[2] M. Cavaliere, O. H. Ibarra, Gh. Păun, O. Egecioglu, M. Ionescu and S. Woodworth. Asynchronous spiking neural P systems. Theoretical Computer Science, 2009, 410(24-25): 2352–2364.

[3] J. E. Hopcroft, R. Motwani and J. D. Ullman. Introduction to automata theory, languages, and computation. Acm Sigact News, 2001, 32(1): 60–65.

[4] O. H. Ibarra, A. Păun and A. Rodríguez-Patón. Sequential SNP systems based on min/max spike number. Theoretical Computer Science, 2009, 410(30-32): 2982–2991.

[5] M. Ionescu, Gh. Păun and T. Yokomori. Spiking neural P systems. Fundamenta Informaticae, 2006, 71(2-3): 279–308.

[6] M. Ionescu, Gh. Păun and T. Yokomori. Spiking neural P Systems with an exhaustive use of rules. International Journal of Unconventional Computing, 2007, 3(2): 974–997.

[7] Z. Lv, Q. Yang, H. Peng, X. Song and J. Wang. Computational power of sequential spiking neural P systems with multiple channels. Journal of Membrane Computing, 2021, 4(3): 270–283.

[8] V. P. Metta, K. Krithivasan and D. Garg. Computability of spiking neural P systems with anti-spikes. New Mathematics and Natural Computation, 2012, 8(3): 283–295.

[9] L. F. Macías-Ramos and M. J. Pérez-Jiménez. Spiking neural P systems with functional astrocytes. International Conference on Membrane Computing, 2012: 228–242.

[10] C. Martín-Vide, Gh. Păun, J. Pazos and A. Rodríguez-Patón. Tissue P systems. Theoretical Computer Science, 2003, 296(2): 295–326.

[11] Gh. Păun. Computing with membranes. Journal of Computer and System Sciences, 2000, 61(1): 108–143.

[12] Gh. Păun. Membrane computing: an introduction. Berlin, Germany: Springer, 2002.

[13] Gh. Păun. Spiking Neural P Systems with Astrocyte-Like Control. Journal of Universal Computer Science, 2007, 13(11): 1707–1721.

[14] H. Peng, Z. Lv, B. Li, X. Luo, J. Wang, X. Song, T. Wang, M. J. Pérez-Jiménez and A. Riscos-Núñez. Nonlinear spiking neural P systems. International Journal of Neural Systems, 2020, 30(10): 2050008.

[15] H. Peng, B. Li, Q. Yang and J. Wang. Multi-focus image fusion approach based on CNP systems in NSCT domain. Computer Vision and Image Understanding, 2021, 210: 103228.

[16] L. Pan and Gh. Păun. Spiking neural P systems with anti-spikes. International Journal of Computers Communications & Control, 2009, 4(3): 273–282.

[17] L. Pan, Gh. Păun, and M. J. Pérez-Jiménez. Spiking neural P systems with neuron division and budding. Science China Information Sciences, 2011, 54(8): 1596–1607.

[18] H. Peng and J. Wang. Coupled neural P systems. IEEE Transactions on Neural Networks and Learning Systems, 2018, 30(6): 1672–1682.

[19] G. Rozenberg and A. Salomaa. The Oxford handbook of membrane computing. England: Oxford University Press, 2010.

[20] G. Rozenberg and A. Salomaa. Handbook of Formal Languages: Volume 3 Beyond Words. Springer Science & Business Media, 2012.

[21] B. Song, K. Li, D. Orellana-Martín, M. J. Pérez-Jiménez and I. Pérez-Hurtado. A survey of nature-inspired computing: membrane computing. ACM Computing Surveys, 2021, 54(1): 1–31.

[22] B. Song, K. Li, D. Orellana-Martín, L. Valencia-Cabrera, and M. J. Pérez-Jiménez. Cell-like P systems with evolutional symport/antiport rules and membrane creation. Information and Computation, 2020, 275: 104542.

[23] B. Song, K. Li and X. Zeng. Monodirectional evolutional symport tissue P systems with promoters and cell division. IEEE Transactions on Parallel and Distributed Systems, 2022, 33(2): 332–342.

[24] B. Song, X. Luo, L. Valencia-Cabrera and X. Zeng. The computational power of cell-like P systems with one protein on membrane. Journal of Membrane Computing, 2020, 4(2): 332–340.

[25] B. Song and L. Pan. Rule synchronization for tissue P systems. Information and Computation, 2021, 281: 104685.

[26] B. Song, L. Pan and M. J. Pérez-Jiménez. Cell-like P systems with channel states and symport/antiport rules. IEEE Transactions on NanoBioscience, 2016, 15(6): 555–566.

[27] T. Song, L. Pan, K. Jiang, B. Song and W. Chen. Normal forms for some classes of sequential spiking neural P systems. IEEE Transactions on NanoBioscience, 2013, 12(3): 255–264.

[28] T. Song, L. Pan and Gh. Păun. Spiking neural P systems with rules on synapses. Theoretical Computer Science, 2014, 529: 82–95.

[29] T. Song, A. Rodríguez-Patón, P. Zheng and X. Zeng. Spiking neural P systems with colored spikes. IEEE Transactions on Cognitive and Developmental Systems, 2017, 10(4): 1106–1115.

[30] X. Song, L. Valencia-Cabrera, H. Peng, J. Wang and M. J. Pérez-Jiménez. Spiking neural P systems with delay on synapses. International Journal of Neural Systems, 2021, 31(1): 2050042.

[31] J. Wang, H. J. Hoogeboom, L. Pan, Gh. Păun and M. J. Pérez-Jiménez. Spiking neural P systems with weights. Neural Computation, 2010, 22(10): 2615–2646.

[32] T. Wu and S. Jiang. Spiking neural P systems with a flat maximally parallel use of rules. Journal of Membrane Computing, 2021, 3(3): 221–231.

[33] T. Wu, Q. Lyu and L. Pan. Evolution-communication spiking neural P systems. International Journal of Neural Systems, 2021, 31(2): 2050064.

[34] T. Wu, L. Pan, Q. Yu and K. C. Tan. Numerical spiking neural P systems. IEEE Transactions on Neural Networks and Learning Systems, 2020, 32(6): 2443–2457.

[35] T. Wu, L. Zhang, Q. Lyu and Y. Jin. Asynchronous spiking neural P systems with local synchronization of rules. Information Sciences, 2022, 588: 1–12.

[36] X. Zhang, B. Luo, X. Fang and L. Pan. Sequential spiking neural P systems with exhaustive use of rules. BioSystems, 2012, 108(1-3): 52–62.

[37] X. Zhang, L. Pan and A. Păun. On the universality of axon P systems. IEEE Transactions on Neural Networks and Learning Systems, 2015, 26(11): 2816–2829.

[38] X. Zhang, B. Wang and L. Pan. Spiking neural P systems with a generalized use of rules. Neural Computation, 2014, 26(12): 2925–2943.

[39] X. Zhang, X. Zeng, B. Luo and L. Pan. On some classes of sequential spiking neural P systems. Neural Computation, 2014, 26(5): 974–997.

[40] X. Zeng, X. Zhang and L. Pan. Homogeneous spiking neural P systems. Fundamenta Informaticae, 2009, 97(1-2): 275–294.